

# Apache Flink

# AdaptiveScheduler

## 核心架构设计文档

—— 基于 Flink 1.15.4 源码深度分析 ——

|      |                                                 |
|------|-------------------------------------------------|
| 模块   | flink-runtime / scheduler / adaptive            |
| 版本   | release-1.15.4                                  |
| 文件数  | 64个Java源文件 (主目录28 + allocator 24 + timeline 12) |
| 核心设计 | 状态机模式 + 声明式资源管理 + 自适应并行度                        |
| 关键特性 | FLIP-160: 资源不足时自动降级启动作业                         |

# 目录

## 第一章 AdaptiveScheduler 整体架构概览

- 1.1 模块组成与文件结构
- 1.2 核心设计思想与 FLIP-160
- 1.3 与 DefaultScheduler 的对比

## 第二章 状态机模式设计

- 2.1 核心状态机全景图
- 2.2 State 接口与类型安全分发
- 2.3 Context 模式设计
- 2.4 StateWith/WithoutExecutionGraph 抽象层

## 第三章 状态转换管理器

- 3.1 五阶段状态机
- 3.2 资源稳定性判定与防抖

## 第四章 作业生命周期核心流程

- 4.1 作业启动流程
- 4.2 自适应扩缩容流程
- 4.3 故障恢复流程

## 第五章 Slot 分配子系统

- 5.1 SlotSharingSlotAllocator 分配流程
- 5.2 三种 SlotMatchingResolver 策略

## 第六章 Rescale Timeline 子系统

- 6.1 FLIP-495 架构
- 6.2 核心数据模型

## 第七章 设计模式总结与关键类说明

- 7.1 设计模式清单
- 7.2 关键类职责
- 7.3 核心方法调用链

# 第一章 AdaptiveScheduler 整体架构概览

## 1.1 模块组成与文件结构

AdaptiveScheduler 位于 flink-runtime 的 org.apache.flink.runtime.scheduler.adaptive 包中，是 Flink 针对弹性资源环境设计的自适应调度器，共 64 个 Java 源文件：

| 目录                  | 文件数 | 核心职责                       |
|---------------------|-----|----------------------------|
| adaptive/ (主目录)     | 28  | 状态机核心：调度器、各状态类、状态转换管理      |
| adaptive/allocator/ | 24  | Slot 分配子系统：分配器、匹配器、共享 Slot |
| adaptive/timeline/  | 12  | 扩缩容时间线：记录和统计 Rescale 事件    |

## 1.2 核心设计思想与 FLIP-160

AdaptiveScheduler 实现了 FLIP-160 的自适应调度器理念。与 DefaultScheduler 要求资源完全满足后才启动不同，AdaptiveScheduler 采用声明式资源管理，能在资源不足时自动降低并行度启动，并在资源变化时动态扩缩容：

- 声明式资源管理：通过 DeclarativeSlotPool 声明资源需求，而非命令式申请 Slot
- 自适应并行度：资源不足时自动计算可用最大并行度，降级启动
- 弹性扩缩容：运行中资源变化时通过 Rescale 重建 ExecutionGraph
- 稳定性判定：StateTransitionManager 多阶段确保资源稳定后才扩缩容

FLIP-160 核心理念：传统调度器要求所有 Slot 就位才启动。AdaptiveScheduler 先声明资源需求，然后根据实际可用 Slot 自动计算并行度。这使 Flink 在 K8s/YARN 弹性环境中不会因个别 Slot 未就绪而阻塞整个作业。

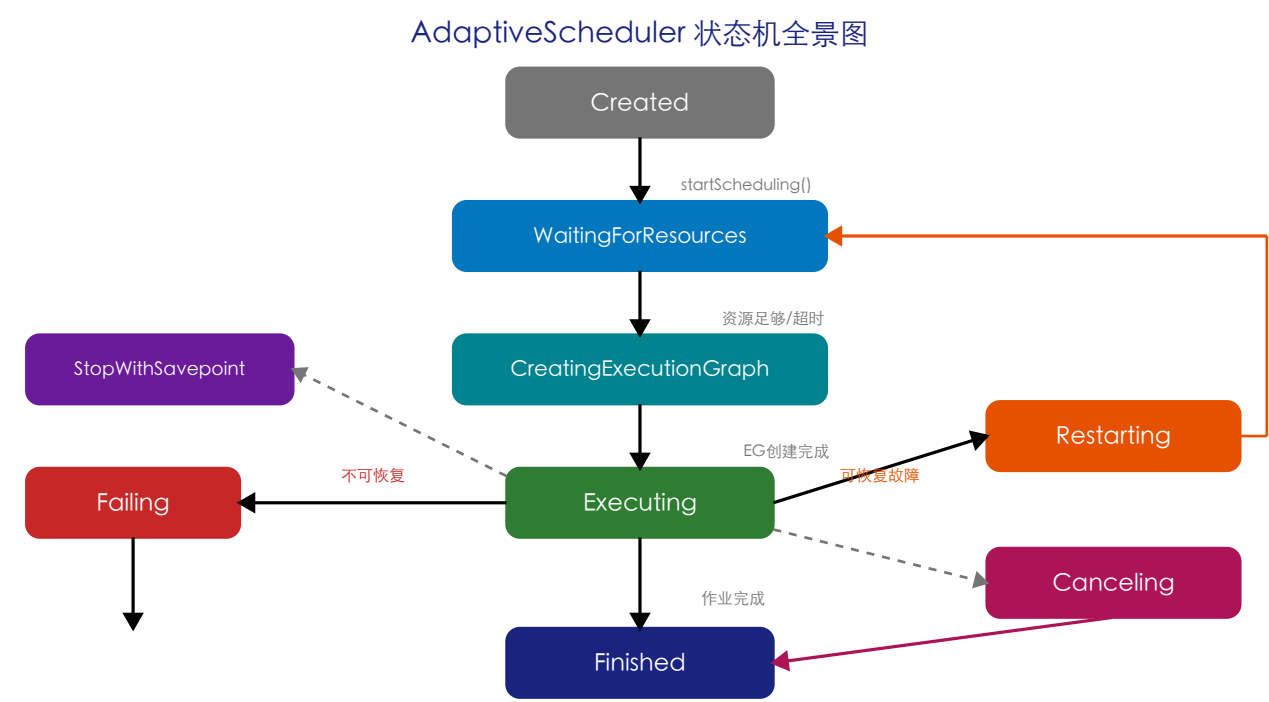
## 1.3 与 DefaultScheduler 的对比

| 特性             | DefaultScheduler | AdaptiveScheduler |
|----------------|------------------|-------------------|
| 资源模型           | 命令式(逐个申请Slot)    | 声明式(声明总需求)        |
| 并行度            | 固定(配置时确定)        | 自适应(运行时调整)        |
| 扩缩容            | 不支持              | 自动(Rescale机制)     |
| ExecutionGraph | 启动前一次性创建         | 可能多次重建            |
| 故障恢复           | Region级别         | 全局级别(整个作业重启)      |
| 适用场景           | 稳定资源环境           | 弹性资源环境(K8s/YARN)  |

## 第二章 状态机模式设计

### 2.1 核心状态机全景图

AdaptiveScheduler 的核心是一个精心设计的有限状态机(FSM)。每个状态封装该阶段的所有行为逻辑，将复杂调度逻辑分解到独立状态类中：



| 状态                     | 继承        | 核心职责                               |
|------------------------|-----------|------------------------------------|
| Created                | WithoutEG | 初始状态，等待startScheduling()           |
| WaitingForResources    | WithoutEG | 声明资源需求，等待Slot到达                    |
| CreatingExecutionGraph | WithoutEG | 异步创建ExecutionGraph(BackgroundTask) |
| Executing              | WithEG    | 部署Task，处理Checkpoint，监听资源变化         |
| Restarting             | WithEG    | 取消Task，等待完成后重新WaitingForResources  |
| Failing                | WithEG    | 不可恢复故障，cancel后→Finished(FAILED)    |
| Canceling              | WithEG    | 用户取消，等待Task取消完成                    |
| Finished               | State     | 终态：完成/失败/取消                        |
| StopWithSavepoint      | WithEG    | 触发Savepoint后停止作业                   |

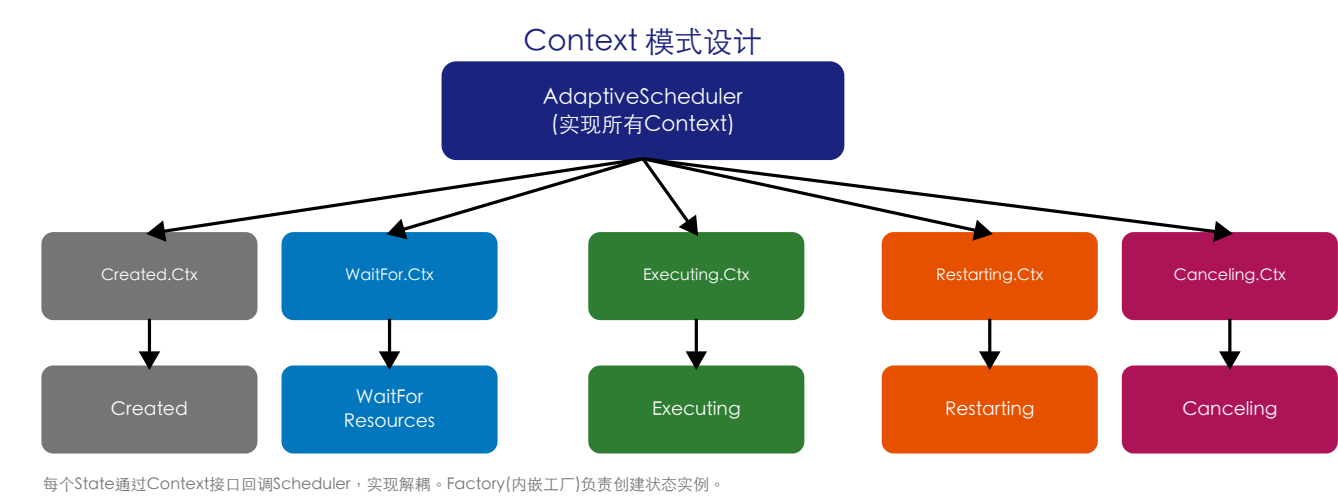
### 2.2 State 接口与类型安全分发

State 接口(198行)定义了类型安全的方法分发机制。RPC调用到达时，通过 tryRun/tryCall/as 三个泛型方法实现安全的状态感知操作：

| 方法        | 签名                                                | 作用                         |
|-----------|---------------------------------------------------|----------------------------|
| tryRun()  | tryRun(Class&lt;T&gt;, Consumer&lt;T&gt;, String) | 当前状态是T类型则执行Consumer        |
| tryCall() | tryCall(Class&lt;T&gt;, FunctionWithException)    | 当前状态是T类型则执行Function返回结果    |
| as()      | as(Class&lt;T&gt;)                                | 安全类型转换，返回Optional&lt;T&gt; |

设计意义：例如 triggerSavepoint() 只在 Executing 状态有效。通过 tryCall(Executing.class, ...) 模式，非 Executing 状态会安全抛出异常而非 ClassCastException，避免大量 if-else 状态检查。

2.3 Context 模式设计



- 解耦：State 不直接依赖 AdaptiveScheduler，只依赖抽象 Context 接口
- ISP：每个 Context 只含该状态需要的方法，符合接口隔离原则
- 可测试：Mock Context 即可测试，无需完整 Scheduler
- Factory：每个 State 内嵌 Factory 类，通过 Context 注入创建实例

2.4 StateWithExecutionGraph / StateWithoutExecutionGraph

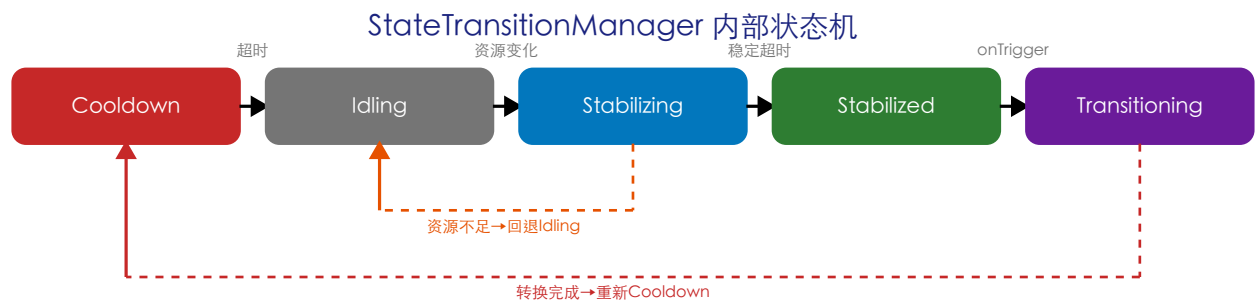
| 基类                                   | 子类                                                           | 特点                                     |
|--------------------------------------|--------------------------------------------------------------|----------------------------------------|
| StateWithoutExecutionGraph<br>(109行) | Created, WaitingForResources, CreatingExecutionGraph         | 不持有EG；对运行时操作返回默认值                      |
| StateWithExecutionGraph<br>(492行)    | Executing, Restarting, Failing, Canceling, StopWithSavepoint | 持有EG；实现Task操作转发、Checkpoint管理、Archive构建 |

StateWithExecutionGraph(492行)核心功能：updateTaskExecutionState()处理Task状态更新、triggerSavepoint()委托CheckpointCoordinator、goToFinished()构建ArchivedExecutionGraph转入终态。所有持有EG的状态共享这些行为。

## 第三章 状态转换管理器

### 3.1 DefaultStateTransitionManager 五阶段状态机

StateTransitionManager(428行)是一个精巧的子状态机，管理资源变化到实际扩缩容之间的时序控制，解决资源频繁波动时避免不必要的EG重建：



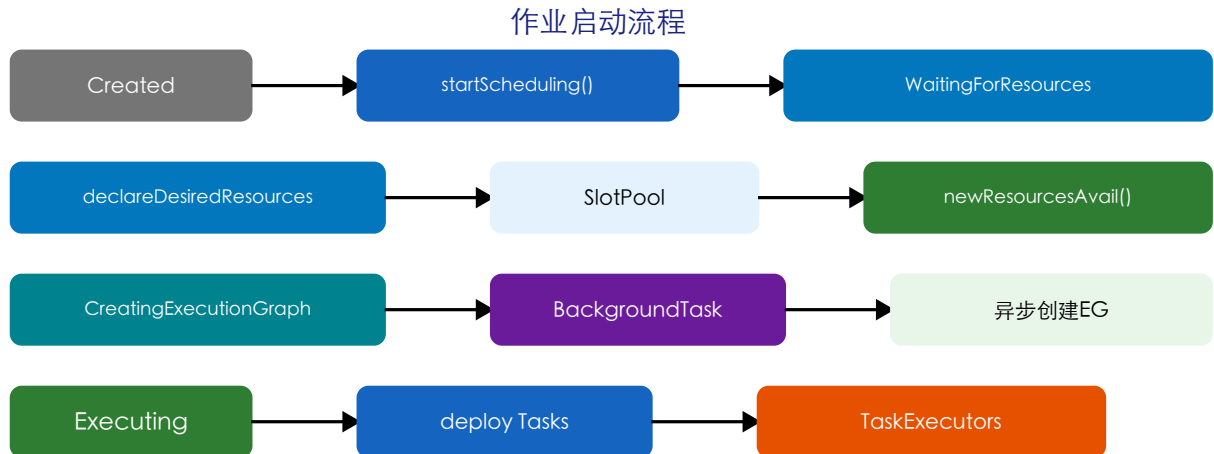
| 阶段            | 触发          | 超时行为        | 逻辑              |
|---------------|-------------|-------------|-----------------|
| Cooldown      | 转换完成        | →Idling     | 防抖冷却期           |
| Idling        | Cooldown超时  | 无           | 空闲等待资源变化        |
| Stabilizing   | 资源变化        | →Stabilized | 等待稳定，新变化重置计时器   |
| Stabilized    | 稳定超时        | 无           | 通知Scheduler执行转换 |
| Transitioning | onTrigger() | →Cooldown   | 执行实际扩缩容         |

防抖效果：K8s环境中Pod扩缩导致Slot频繁变化。Cooldown+Stabilizing两阶段确保资源稳定后才真正Rescale。典型配置：cooldown=10s, stabilization=10s。

## 第四章 作业生命周期核心流程

### 4.1 作业启动流程

作业启动遵循 Created → WaitingForResources → CreatingExecutionGraph → Executing 的状态链：



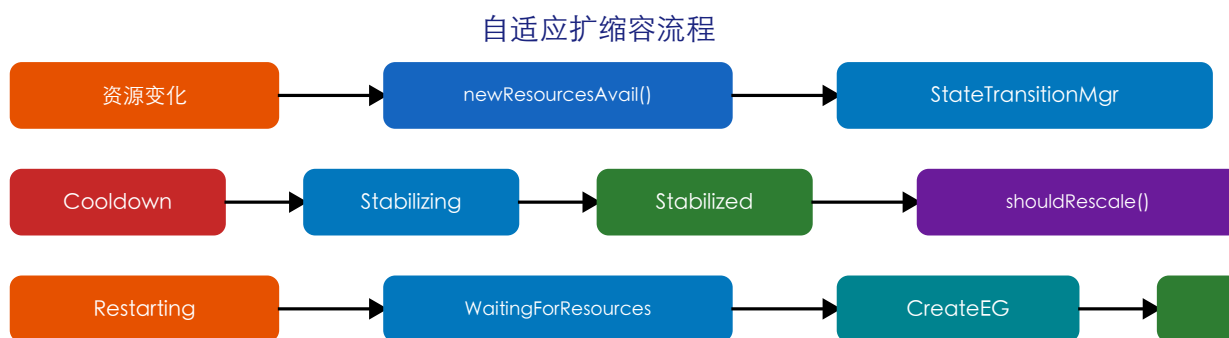
- startScheduling() — Created 转入 WaitingForResources，声明资源需求
- newResourcesAvailable() — 检查可用Slot，达到最小要求则创建EG
- 超时机制 — resourceWaitingTimeout到期，以当前可用资源强制启动
- BackgroundTask — 异步创建EG+恢复Checkpoint，支持abort安全取消
- Executing入口 — deploy所有Task，启动CheckpointScheduler

BackgroundTask设计：支持链式调用(andThen)。状态转换时abort()安全取消后续操作，避免竞态条件。

### 4.2 自适应扩缩容流程

Executing 状态收到资源变化时，通过 StateTransitionManager 判断是否需要扩缩容：

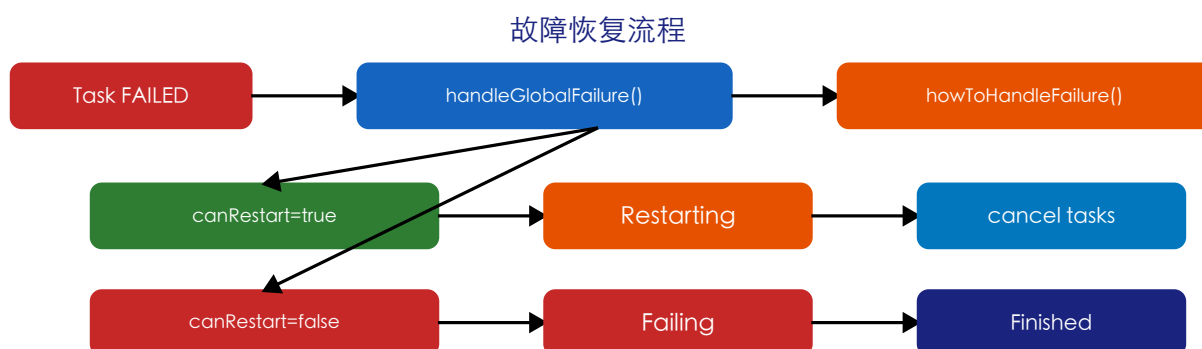




- Step 1 : SlotAllocator.determineParallelism() — 计算新并行度
- Step 2 : 比较新旧 VertexParallelism — 并行度变化则 rescale
- Step 3 : 检查 SlotsUtilization — 利用率可改善也触发 rescale
- Step 4 : Executing → Restarting → WaitingForResources → CreatingEG → Executing

## 4.3 故障恢复流程

AdaptiveScheduler 采用全局故障恢复，以整个作业为恢复粒度：

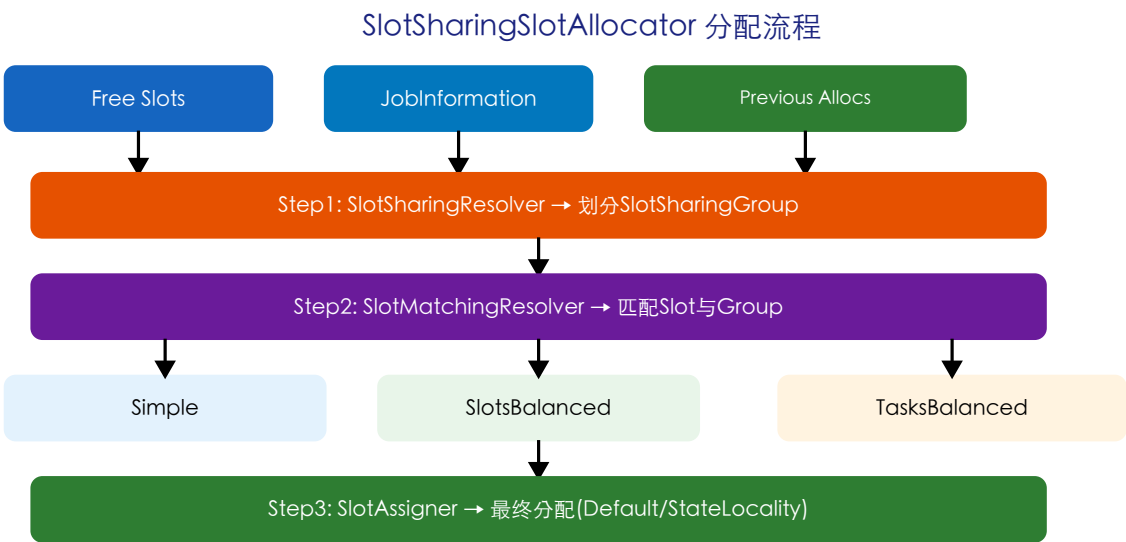


- handleGlobalFailure() — 入口，接收Throwable
- howToHandleFailure() — 咨询RestartBackoffTimeStrategy
- 可恢复：→ Restarting(cancel tasks) → WaitingForResources
- 不可恢复：→ Failing(cancel tasks) → Finished(FAILED)
- Restarting细节：cancel所有Task，终态后根据backoffTime延迟重启

# 第五章 Slot 分配子系统 (allocator)

## 5.1 SlotSharingSlotAllocator 分配流程

allocator 子目录24个文件，核心 SlotSharingSlotAllocator(458行) 实现三阶段 Slot 分配：



## 5.2 三种 SlotMatchingResolver 策略

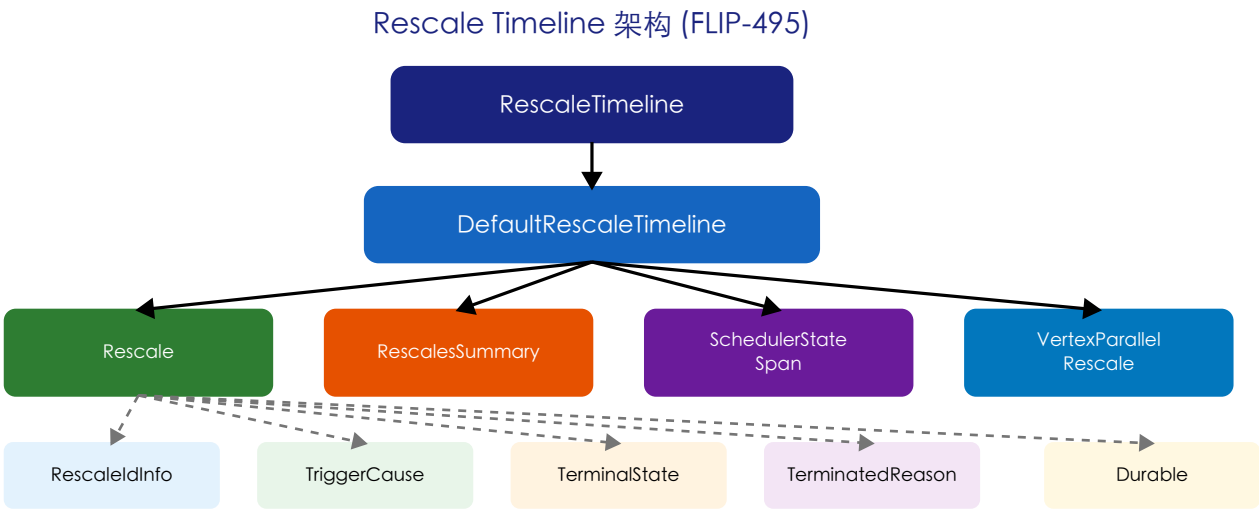
| 策略            | 实现类                        | 算法            | 适用场景     |
|---------------|----------------------------|---------------|----------|
| Simple        | SimpleSlotMatchingResolver | 顺序FIFO分配      | 简单快速     |
| SlotsBalanced | SlotsBalancedSlotMR        | 均衡分配Slot到不同TE | 分散负载     |
| TasksBalanced | TasksBalancedSlotMR        | 均衡Task到不同TE   | Task均匀分布 |

allocator 三层策略：SlotSharingResolver(解析Group需求) → SlotMatchingResolver(匹配Slot到Group) → SlotAssigner(分配Task到Slot)。三层分离使每层可独立替换测试。StateLocalitySlotAssigner 通过评分机制最大化状态本地性。

## 第六章 Rescale Timeline 子系统

### 6.1 FLIP-495 架构

timeline 子目录12个文件，实现FLIP-495的扩缩容时间线记录，用于监控和调试：



### 6.2 核心数据模型

| 类名                       | 行数  | 职责                          |
|--------------------------|-----|-----------------------------|
| RescaleTimeline          | 107 | 顶层接口                        |
| DefaultRescaleTimeline   | 148 | 默认实现，维护Rescale列表            |
| Rescale                  | 468 | 核心类：记录单次扩缩容完整生命周期           |
| RescalesSummary          | 137 | 统计摘要：聚合耗时统计                 |
| SchedulerStateSpan       | 124 | 调度器状态时间跨度                   |
| VertexParallelismRescale | 161 | 顶点并行度变化记录                   |
| SlotSharingGroupRescale  | 160 | Slot共享组变化记录                 |
| TriggerCause             | 38  | 触发原因：SCALING_UP/DOWN/FORCED |
| TerminalState            | 48  | 终态：DONE/FAILED/DISCARDED    |

# 第七章 设计模式总结与关键类说明

## 7.1 设计模式清单

| 模式               | 应用位置                    | 实现                                  |
|------------------|-------------------------|-------------------------------------|
| State Pattern    | AdaptiveScheduler核心     | 9个状态类封装不同阶段行为                       |
| Factory Pattern  | 每个State类                | 内嵌Factory类注入Context创建实例             |
| Strategy Pattern | allocator子系统            | 3种MatchingResolver+2种Assigner       |
| Context Pattern  | State-Scheduler解耦       | 每状态定义Context，Scheduler统一实现          |
| Template Method  | StateWithExecutionGraph | 基类定义流程，子类实现具体行为                     |
| Observer Pattern | ResourceListener        | WaitingForResources/Executing监听资源变化 |
| Adapter Pattern  | ForwardEdgesAdapter     | 适配ExecutionTopology为正向边查询           |

## 7.2 关键类职责说明

| 类名                            | 行数   | 核心职责                          |
|-------------------------------|------|-------------------------------|
| AdaptiveScheduler             | 1792 | 核心调度器，实现SchedulerNG和所有Context |
| SlotSharingSlotAllocator      | 458  | Slot分配核心，组合Resolver+Assigner  |
| StateWithExecutionGraph       | 492  | 持有EG的状态基类，共享Task和CP管理         |
| DefaultStateTransitionManager | 428  | 五阶段子状态机，资源稳定性判定               |
| Executing                     | 389  | 核心执行状态，deploy/CP/资源监听         |
| StopWithSavepoint             | 363  | 协调SP触发、完成和停止                  |
| Rescale                       | 468  | 扩缩容完整生命周期记录                   |
| StateLocalitySlotAssigner     | 216  | 基于状态本地性优化的分配器                 |

## 7.3 核心方法调用链

以下是 AdaptiveScheduler 最重要的调用链，覆盖作业完整生命周期：

### 调用链 1：作业启动

```
AdaptiveScheduler.startScheduling() → Created.onLeave() → goToWaitingForResources() →
WaitingForResources.onNewResourcesAvailable() → hasDesiredResources()/hasEnoughResources() →
goToCreatingExecutionGraph() → BackgroundTask.run() → handleExecutionGraphCreation() →
goToExecuting() → Executing.onEnter() → deploy all Tasks
```

### 调用链 2：自适应扩缩容

```
Executing.newResourcesAvailable() → StateTransitionManager.onChange() → Stabilizing → Stabilized →  
onSwitchToState() → shouldRescale() → SlotAllocator.determineParallelism() → goToRestarting() →  
Restarting.cancel() → onGloballyTerminalState() → goToWaitingForResources() → [新并行度创建EG]
```

### 调用链 3: 故障恢复

```
Executing.handleGlobalFailure() → howToHandleFailure() → RestartBackoffTimeStrategy.canRestart() →  
[true] goToRestarting(backoffTime) → Restarting → WaitingForResources → [false] goToFailing() →  
Failing.cancel() → onGloballyTerminalState() → goToFinished(FAILED)
```

### 调用链 4: StopWithSavepoint

```
Executing.stopWithSavepoint() → goToStopWithSavepoint() → StopWithSavepoint.onEnter() →  
checkpointCoordinator.triggerSavepoint() → handleSavepointCompletion() → 等待Tasks完成 →  
goToFinished(savepointPath)
```